M-HIBE ???????????

????

????? D={p1,...,pn}??? d ?????????? L bit ??????????? ?=(?1,...,?d)?????????? ?=(X,Y,Z)?? ??

????????

```
Algorithm 1: Offline Empty Parent Key Generation

Input:
    Dataset D, dimension order pi, bit length L, master secret key MSK
Output:
    Global empty parent key set K

1. Initialize an empty prefix index T

2. For each point p in D:
       For i = 1 to d:
           parent = prefixes of p on dimensions pi_1,...,pi_{i-1}
           value  = p_{pi_i}
           Insert value into T[parent, pi_i]

3. K = empty set

4. For each indexed parent node parent in T:
       i = next dimension after parent
       S = T[parent, pi_i]
       E = CanonicalEmptyCover([0, 2^L - 1] - S)

       For each empty prefix e in E:
           pattern = parent || e || wildcard for all remaining dimensions
           sk = M-HIBE.KeyGen(MSK, pattern)
           K = K union {(pattern, sk)}

5. Return K
```

?????

?????? X?Y?Z????????????IX[X-prefix] ??? X ??????? Y ????IXY[X-prefix,Y-prefix] ??? (X,Y) ??????? Z ????

```
Algorithm 2: 3D Empty Parent Key Generation

Input:
    3D dataset D, order X -> Y -> Z, master secret key MSK
Output:
    Global empty parent key set K

1. Build two indexes:
       IX[X-prefix]              = set of Y values under this X-prefix
       IXY[X-prefix,Y-prefix]  = set of Z values under this (X,Y)-prefix

2. K = empty set

3. For each X-prefix px:
       SY = IX[px]
       EY = CanonicalEmptyCover(FullYDomain - SY)

       For each empty Y-prefix py in EY:
           pattern = (px, py, *)
           sk = M-HIBE.KeyGen(MSK, pattern)
```

```
            K = K union {(pattern, sk)}

4. For each indexed pair (px, py):
       SZ = IXY[px, py]
       EZ = CanonicalEmptyCover(FullZDomain - SZ)

       For each empty Z-prefix pz in EZ:
           pattern = (px, py, pz)
           sk = M-HIBE.KeyGen(MSK, pattern)
           K = K union {(pattern, sk)}

5. Return K
```

????????

??

```
Algorithm 3: Query-Scoped Empty Key Derivation

Input:
    Query range Q, global empty parent keys K
Output:
    Query-scoped empty key set KQ

1. KQ = empty set

2. For each (pattern, sk) in K:
       I = pattern intersect Q

       If I is empty:
           continue

       C = CanonicalCover(I)

       For each child pattern c in C:
           child_sk = Derive(sk, c)
           KQ = KQ union {(c, child_sk)}

3. Optionally run SetCover(KQ) to remove redundant keys

4. Return KQ
```

?????

??? X?Y?Z?????????? O(L) ? X ??? O(L?) ? (X,Y) ??????????? O(nL?)?? query-scoped empty keys ???